

COMP7940 Cloud Computing

Lab 5: GitHub Actions with AWS EC2 Deployment

Name: ZHAO Zhan
Student ID: 25482351
GitHub: @Extious

March 15, 2026

1. Workflow Overview

A GitHub Actions workflow was configured to automate the deployment of the Telegram chatbot to an AWS EC2 instance. The workflow file `.github/workflows/deploy.yml` is triggered on every push to the `main` branch. Three repository secrets—`EC2_HOST`, `EC2_USER`, and `EC2_KEY`—were configured in the GitHub repository settings to securely provide the SSH credentials needed for deployment.

2. Write-Up Questions

Q1: Identify the command in `deploy.yml` that deploys the chatbot

The deployment is carried out by the `ssh` command in the “Deploy application” step of the workflow. This command establishes an SSH connection to the EC2 instance using the credentials stored in GitHub Secrets and executes a series of remote commands:

```
ssh -o StrictHostKeyChecking=no ${ secrets.EC2_USER }}@${ secrets.EC2_HOST }} << 'EOF'
cd ~/comp7940-lab/lab5
git pull origin main
pkill -f "python chatbot.py" || true
nohup ~/comp7940-lab/venv/bin/python chatbot.py > app.log 2>&1 &
EOF
```

Specifically, the `ssh` command connects to the EC2 instance, navigates to the project directory, pulls the latest code from the `main` branch using `git pull`, terminates the previously running chatbot process (if any), and restarts the chatbot in the background using `nohup`. The `-o StrictHostKeyChecking=no` flag suppresses the host-key verification prompt, which is necessary for non-interactive CI/CD environments.

Q2: CI/CD data flow diagram

Figure 1 illustrates the connections and data flow among the three key components in the CI/CD pipeline.

1. **Step 1 — git push:** The developer commits changes locally and pushes them to the remote GitHub repository on the `main` branch.

2. **Step 2 — Workflow trigger:** The push event automatically triggers the GitHub Actions workflow defined in `deploy.yml`. The workflow runner (an `ubuntu-latest` virtual machine) checks out the repository code and loads the SSH private key from GitHub Secrets.
3. **Step 3 — SSH deployment:** The runner establishes an SSH connection to the AWS EC2 instance using the stored credentials. On the EC2 instance, it pulls the latest code, kills any existing chatbot process, and starts the updated chatbot in the background.

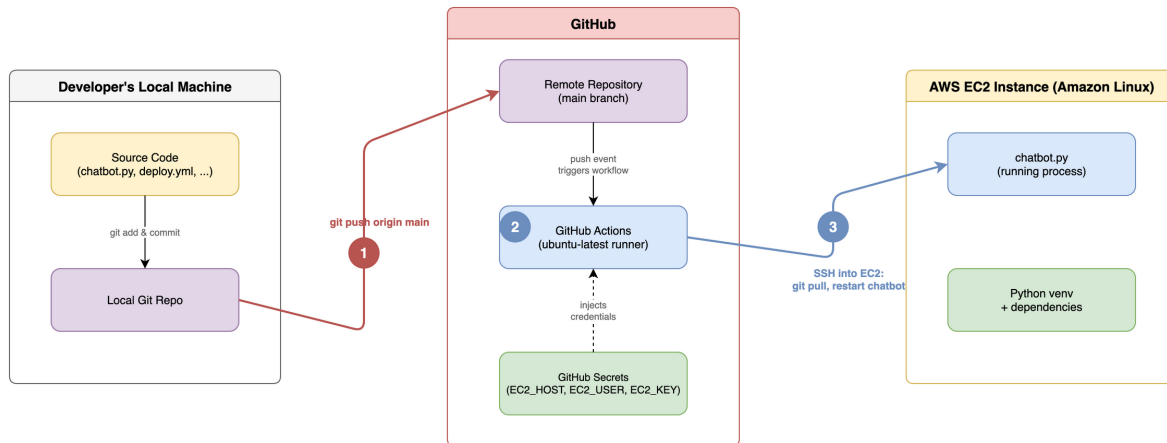


Figure 1: CI/CD data flow between the local repository, GitHub, and the AWS EC2 instance.

Q3: Why storing passwords in `config.ini` is insecure, and how GitHub Secrets improve security

Storing sensitive credentials (such as API keys and access tokens) directly in `config.ini` is insecure for several reasons:

1. **Version control exposure.** If `config.ini` is committed to a Git repository, the credentials become part of the repository history. Even if the file is later removed, the secrets remain recoverable from previous commits. If the repository is public, the credentials are exposed to anyone on the internet.
2. **Plain-text storage.** The file stores secrets as plain text on disk. Anyone with access to the filesystem—whether through a compromised machine, a shared development environment, or a misconfigured server—can read the credentials directly.
3. **No access control.** There is no built-in mechanism to restrict who can view or modify the contents of a plain configuration file. Any collaborator with repository access can read the secrets.

GitHub Secrets address these issues by providing a secure, encrypted storage mechanism [1]:

- **Encryption at rest.** Secrets are encrypted using Libsodium sealed boxes before being stored on GitHub's servers. They are only decrypted in memory on the runner during workflow execution and are never written to disk in plain text.

- **Masked in logs.** GitHub Actions automatically redacts secret values from workflow logs. Even if a command accidentally prints a secret, it appears as `***` in the log output.
- **Access control.** Only repository administrators can create or update secrets. Workflow files can reference secrets, but no user (including administrators) can view the secret values after they are saved.
- **Not stored in source code.** Secrets are managed outside the repository codebase, which eliminates the risk of accidental commits and exposure in version history.

Q4: GitHub Actions run screenshots

Figure 2 shows the detailed logs of a successful GitHub Actions workflow run for the “AWS EC2 Deploy” workflow. The run consists of several steps:

- **Set up job:** The runner environment (`ubuntu-latest`) is initialized.
- **Checkout code:** The repository is cloned using `actions/checkout@v3`.
- **Setup SSH:** The SSH agent is started and the private key from `EC2_KEY` is loaded via `webfactory/ssh-agent@v0.7.0`.
- **Deploy application:** The runner SSHs into the EC2 instance, pulls the latest code, and restarts the chatbot process. The logs show a successful connection to the Amazon Linux 2023 instance and a `git pull` completing with a fast-forward merge.
- **Post-job cleanup:** The SSH agent is stopped and temporary credentials are removed.

Figure 3 shows the application logs (`app.log`) from the EC2 instance after deployment. The chatbot initializes successfully: it loads configuration, connects to the Telegram API, registers the message handler, and begins polling for updates. The repeated `getUpdates` HTTP requests confirm that the bot is running and actively listening for new messages.

References

- [1] GitHub. Github actions documentation – using secrets in github actions. Available at <https://docs.github.com/en/actions/security-for-github-actions/security-guides/using-secrets-in-github-actions>, 2026. Accessed: 2026-03-15.

```
Deploy to EC2
succeeded 3 minutes ago in 8s

🔍 Search logs

Set up job 1s
1 Current runner version: '2.332.0'
2 ▶ Runner Image Provisioner
9 ▶ Operating System
13 ▶ Runner Image
18 ▶ GITHUB_TOKEN Permissions
22 Secret source: Actions
23 Prepare workflow directory
24 Prepare all required actions
25 Getting action download info
26 Download action repository 'actions/checkout@v3' (SHA: f43a0e5ff2bd294895638e18286ca9a3d1956744)
27 Download action repository 'webfactory/ssh-agent@v0.7.0' (SHA: 836c84ec59a8e7bc8eabc79988384e0567561ee2)
28 Complete job name: Deploy to EC2

Checkout code 1s
1 ▶ Run actions/checkout@v3
14 Syncing repository: Extious/comp7940-lab
15 ▶ Getting Git version info
19 Temporarily overriding HOME='/home/runner/work/_temp/25413f07-1055-416e-bc45-8b908f1d8c7a' before making global git config changes
20 Adding repository directory to the temporary git global config as a safe directory
21 /usr/bin/git config --global --add safe.directory /home/runner/work/comp7940-lab/comp7940-lab
22 Deleting the contents of '/home/runner/work/comp7940-lab/comp7940-lab'
23 ▶ Initializing the repository
40 ▶ Disabling automatic garbage collection
42 ▶ Setting up auth
48 ▶ Fetching the repository
122 ▶ Determining the checkout info
123 ▶ Checking out the ref
127 /usr/bin/git log -1 --format="%h"
128 '8484fd2623871cfe8aa2a3bd9428dbf93ce63a86'

Setup SSH 0s
1 ▶ Run webfactory/ssh-agent@v0.7.0
5 Adding GitHub.com keys to /home/runner/.ssh/known_hosts
6 Starting ssh-agent
7 SSH_AUTH_SOCK=/tmp/ssh-4yTB120xRRUF/agent.2251
8 SSH_AGENT_PID=2252
9 Adding private key(s) to agent
10 Identity added: (stdin) ((stdin))
11 Key(s) added:
12 2848 SHA256:NeIvyy0UMK99HUM9Cbb8hAn5snM/r1dSulqEG6ALdc (stdin) (RSA)
13 Configuring deployment key(s)
14 Comment for (pubLic key 'ssh-rsa AAAAB3NzaC1yc2EAAAADAQABAAQNCnRhfsojxs8rcjwoGzC24sRoz2zK4n4enAsxmKFM-r0sKJT2zeA9USMBrfGhnFq7d/JN00HTnUvU1e78BdJ/Adnt0ECt6WwP0u0j3fCgAgtL2gr3+0v+jdGXY1g9HCvLIRYPCxP+GMUZHfY1bzFT8fXA98ed3QJNkzKdfjvAZ0ubeFHYV6Jl+k9YALXfM0685yzRvC7neIRYpIpnlpn0TSFW9+fldPFF0HGfU90V0ertu61fVkuIG0hXp2HqfICB)/e6Zgm9L0oz/hL/zyf0DU3fVAwRw0fSNNSGr3aICMhKoPLAMBgIvyJNTpVUj2HhKZHAJhMX (stdin)' does not match GitHub URL pattern. Not treating it as a GitHub deploy key.

Deploy application 4s
1 ▶ Run # -o StrictHostKeyChecking=no skips the host-key verification prompt on first connect
17 Pseudo-terminal will not be allocated because stdin is not a terminal.
18 Warning: Permanently added '***' (ED25519) to the list of known hosts.
19
20      #_
21  ~\_  ###   Amazon Linux 2023
22  ~\_  \_####\
23  ~\_  \_###\
24  ~\_  \_###\
25  ~\_  \_###\
26  ~\_  \_###\
27  ~\_  \_###\
28  ~\_  \_###\
29 From https://github.com/Extious/comp7940-lab
30 * branch      main              -> FETCH_HEAD
31 4324578..8484fd2 main          -> origin/main
32 Updating 4324578..8484fd2
33 Fast-forward

Post Setup SSH 0s
1 Post job cleanup.
2 Stopping SSH agent
3 unset SSH_AUTH_SOCK;
4 unset SSH_AGENT_PID;
5 echo Agent pid 2252 killed;

Post Checkout code 0s
1 Post job cleanup.
2 /usr/bin/git version
3 git version 2.33.0
4 Temporarily overriding HOME='/home/runner/work/_temp/567fd560-19ed-4694-ab36-17ada4aceeb' before making global git config changes
5 Adding repository directory to the temporary git global config as a safe directory
6 /usr/bin/git config --global --add safe.directory /home/runner/work/comp7940-lab/comp7940-lab
7 /usr/bin/git config --local --name-only --get-regexp core.sshCommand
8 /usr/bin/git submodule foreach --recursive sh -c "git config --local --name-only --get-regexp 'core.sshCommand' && git config --local --unset-all 'core.sshCommand' || :;"
9 /usr/bin/git config --local --name-only --get-regexp http.https://github.com/.extraheader
10 http.https://github.com/.extraheader
11 /usr/bin/git config --local --unset-all http.https://github.com/.extraheader
12 /usr/bin/git submodule foreach --recursive sh -c "git config --local --name-only --get-regexp 'http.https://github.com/.extraheader' && git config --local --unset-all 'http.https://github.com/.extraheader' || :;"

Complete job 0s
1 Cleaning up orphan processes
2 Warning: Node.js 20 actions are deprecated. The following actions are running on Node.js 20 and may not work as expected: actions/checkout@v3, webfactory/ssh-agent@v0.7.0. Actions will be forced to run with Node.js 24 by default starting June 2nd, 2026. Please check if updated versions of these actions are available that support Node.js 24. To opt into Node.js 24 now, set the FORCE_JAVASCRIPT_ACTIONS_TO_NODE24=true environment variable on the runner or in your workflow file. Once Node.js 24 becomes the default, you can temporarily opt out by setting ACTIONS_ALLOW_USE_UNSECURE_NODE_VERSION=true. For more information see: https://github.blog/changelog/2025-09-19-deprecation-of-node-20-on-github-actions-runners/
```

Figure 2: GitHub Actions workflow run logs for the “AWS EC2 Deploy” workflow, showing all steps completed successfully.

```

[[ec2-user@ip-172-31-38-2 lab5]$ cat app.log
2026-03-15 06:17:43,535 - root - INFO - INIT: Loading configuration...
2026-03-15 06:17:43,536 - root - INFO - INIT: Connecting the Telegram bot...
2026-03-15 06:17:43,600 - root - INFO - INIT: Registering the message handler...
2026-03-15 06:17:43,600 - root - INFO - INIT: Initialization done!
2026-03-15 06:17:44,128 - httpx - INFO - HTTP Request: POST https://api.telegram.org/bot8708909924:AAE4TCMFxVF-_ezw3iA2-bIxSMwiG3YGj1s/ge
tMe "HTTP/1.1 200 OK"
2026-03-15 06:17:44,305 - httpx - INFO - HTTP Request: POST https://api.telegram.org/bot8708909924:AAE4TCMFxVF-_ezw3iA2-bIxSMwiG3YGj1s/de
leteWebhook "HTTP/1.1 200 OK"
2026-03-15 06:17:44,306 - telegram.ext.Application - INFO - Application started
2026-03-15 06:17:54,828 - httpx - INFO - HTTP Request: POST https://api.telegram.org/bot8708909924:AAE4TCMFxVF-_ezw3iA2-bIxSMwiG3YGj1s/ge
tUpdates "HTTP/1.1 200 OK"
2026-03-15 06:18:05,003 - httpx - INFO - HTTP Request: POST https://api.telegram.org/bot8708909924:AAE4TCMFxVF-_ezw3iA2-bIxSMwiG3YGj1s/ge
tUpdates "HTTP/1.1 200 OK"
2026-03-15 06:18:15,186 - httpx - INFO - HTTP Request: POST https://api.telegram.org/bot8708909924:AAE4TCMFxVF-_ezw3iA2-bIxSMwiG3YGj1s/ge
tUpdates "HTTP/1.1 200 OK"
2026-03-15 06:18:25,361 - httpx - INFO - HTTP Request: POST https://api.telegram.org/bot8708909924:AAE4TCMFxVF-_ezw3iA2-bIxSMwiG3YGj1s/ge
tUpdates "HTTP/1.1 200 OK"

```

Figure 3: Application logs from the EC2 instance showing successful chatbot initialization and polling.